TrainTurn: Railway Platform Allocation and Delay Minimisation with Masked Double Deep Q-Learning

Siddarth Modugu (*B24CM1057*) Siddhant Gahankari (*B24CM1058*)
GitHub: https://github.com/siddarth777/TrainTurn

I. Problem

When trains run late, platforms clog and delay spreads, hurting high-priority trains most. A controller sees the outer-signal queue, the platforms in use and noisy arrival forecasts, and must give the head train a safe platform or hold it. Each choice decides which platforms are free for trains still minutes away, so the consequences are delayed and stochastic: a natural fit for reinforcement learning.

II. Environment and Simulation Dynamics

Simulator. Trains run physically in Eclipse SUMO 1.28.0 [4] through `libsumo`, wrapped in an original Gymnasium [5] environment. The generated two-corridor junction has 8 platforms of 24, 18 or 12 coaches with rail signals; platform 8 is a terminal platform. SUMO advances in 1 s physical steps inside 1 min dispatch ticks.

Traffic. An episode is one 1,440 min day of 128 trains in five classes with priority weights w: premium 5, superfast 4, express 3, freight 1.5, passenger 1. The nominal timetable, with peak bursts and planned platforms, is built by reserving routes and platforms, so it is delay-free when nothing goes wrong (checked in SUMO).

Disturbances. All randomness is drawn from the day's seed before any policy acts, so every policy faces identical days. Incoming delays mix 65% on time, 30% $\mathrm{LogN}(1.8, 0.6)$ and 5% $\mathrm{LogN}(3.8, 0.9)$ min, scaled by a severity of 0.7. Corridor-wide shocks, dwell overruns and noisy forecasts, whose error shrinks as a train approaches, are added on top.

Operation and safety. Arrivals join an outer queue (capacity 2), with overflow waiting upstream, served first come, first served. Safety holds at three layers: the action mask, interlocking reservations (one entry and one exit movement at a time, 1 min route and 2 min platform clearance) and SUMO's rail signals. Teleports are disabled, and a collision aborts the episode. An episode ends when every train has departed, and is truncated at the horizon.

III. RL Formulation

A decision occurs only when a train heads the outer queue, and it lasts $d_t \geq 1$ minutes until the next one, so the problem is a semi-MDP [3].

State space $\mathcal{S}$. $s \in [-1, 1]^{229}$ is a normalised, zero-padded vector with N=8 platform slots and K=6 forecasts (Table I):

$$\dim \mathcal{S} = 10N + (12 + N) + K(12 + N) + 9 = 229. \quad (1)$$

TABLE I
Observation layout (N=8, K=6)

Block	Size	Features (normalised)
Platform $\times N$	80	present, occupied, time to release, occupant priority, closed, route free, length, electrified, through, clearance
Current train	20	priority, waiting, length, dwell, type, corridors, planned platform, observed delay, N compatibility flags
Forecast $\times K$	120	ETA, estimated delay, uncertainty, priority, dwell, length, type, corridors, plan, N compatibility flags
Global	9	time of day, queues, free platforms, remaining trains, horizon progress, occupancy

It is a partial observation: true future arrivals and hidden dwell overruns are never exposed (unit tests check this).

Action space $\mathcal{A}$. $\mathcal{A} = \{0, \ldots, 8\}$ (`Discrete(9)`): a=0 holds the head train for at least one minute, and a=p assigns platform p. A mask $\mathcal{M}(s)$ allows p only if the platform exists and is compatible (length, electrification, through/terminal service, corridor), free, past its clearance and open, and its entry route is free. After 30 min of waiting, hold is masked whenever a safe platform exists, a safety rule against indefinite holding. Every baseline receives the same mask.

Reward $\mathcal{R}$. For each simulated minute m:

$$r_m = -\tfrac{1}{\kappa}\big(\alpha W_m + \beta O_m + \chi U_m + \delta P_m + \eta H_m\big). \quad (2)$$

$W_m = \sum_{i \in Q_m} w_i$ sums the priorities of trains that are waiting: at the outer signal, upstream, or ready to leave but blocked. O_m and U_m count trains in the outer and upstream queues. P_m=1 in the minute a train is assigned a platform other than its planned one, and H_m=1 in the minute the agent holds while a safe platform is free; both are charged once per decision. We use α=1, β=0.05, χ=0.2, δ=2, η=0.2 and κ=10. The incentives work as follows: priority weighting makes a premium minute cost five times a passenger minute; the queue terms penalise spillback; δ discourages platform changes that confuse passengers; η discourages idle holding. Incoming lateness is never charged, so only delay the station itself causes is penalised. A decision earns $R_t = \sum_{j=0}^{d_t - 1} \gamma^j r_{t,j}$, with $\gamma = 0.99$ per minute.

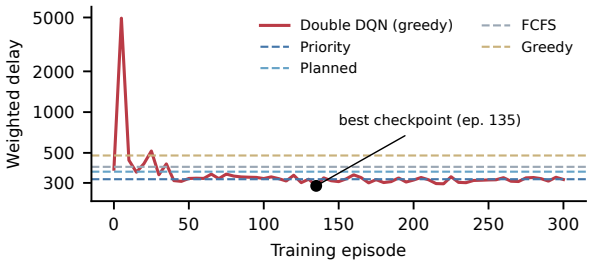


Fig. 1. Greedy validation delay during training (log scale). Dashed lines: heuristics on the same two validation days.

IV. Algorithm and Baselines

Choice. We implement Double DQN [1], [2] from scratch in PyTorch. DQN suits the small, discrete, hard-masked action set; off-policy replay reuses costly SUMO transitions; and Double Q-learning reduces the overestimation that noisy delays cause. Q_θ is a 229–128–128–9 ReLU network (47,113 parameters). With masked online selection $a^* = \arg\max_{a \in \mathcal{M}(s')} Q_\theta(s', a)$, the target is

$$y_t = R_t + \gamma^{d_t} Q_{\theta^-}(s', a^*)\,(1 - \mathrm{term}_t). \tag{3}$$

Episodes cut off at the horizon still bootstrap.

Training. Huber loss with Adam (learning rate 3×10^{-4}), batch 64 and a 50k replay buffer that stores the next mask and d_t. The agent collects 1,024 warm-up transitions, then makes one update per decision; the target network is synchronised every 100 updates and gradients are clipped at 10. Exploration is masked ϵ-greedy, with ϵ falling from 1 to 0.05 over 15k decisions. We trained for 300 episodes (seeds 42–341), validating every 5 episodes on 2 fixed days and keeping the best checkpoint. This took 62,566 decisions and 61,543 updates in 18.5 min on an RTX 3050 laptop GPU.

Baselines and protocol. Four heuristic baselines, used only for comparison, see the same observation and mask, and none can reorder the queue: *FCFS* takes the lowest-numbered feasible platform; *Planned* takes the timetabled platform, otherwise the first feasible one; *Priority* keeps platforms free for imminent high-priority forecasts and prefers the shortest that fits; *Greedy* is a one-step lookahead over the forecast trains. We test on 100 held-out days (seeds 200000–200099), disjoint from the training and validation seeds and shared by all policies. We report 95% bootstrap intervals (2,000 resamples) and paired per-day differences.

V. Results

Learning (Fig. 1). After only 275 updates (episode 5), the network held trains excessively (452 discretionary holds). From about episode 40 it tracked the best heuristic, scoring below it at 27 of 60 validation points. The best checkpoint (episode 135) scored 285.2, against 378.7 untrained and 319.4 for priority; with only two noisy validation days, we judge it on held-out days.

Synthetic test (Table II, Fig. 2a). The DQN has the lowest delay, every paired interval excludes zero, and it beats priority

TABLE II
100 HELD-OUT DAYS (8 PLATFORMS, 128 TRAINS): DAILY WEIGHTED STATION DELAY (EACH TRAIN'S STATION-CAUSED WAITING MINUTES TIMES ITS WEIGHT w, SUMMED; TRAIN-MIN), PAIRED DIFFERENCE DQN−POLICY, AND PLATFORM CHANGES PER DAY.

Policy	Delay [95% CI]	DQN−policy [95% CI]	Changes
Double DQN	**322.1** [308, 338]	–	85.7
Priority	346.1 [331, 363]	−24.1 [−30, −18]	89.8
Planned	374.0 [358, 391]	−51.9 [−59, −45]	18.2
FCFS	398.6 [382, 417]	−76.5 [−84, −69]	64.2
Greedy	515.8 [483, 551]	−193.7 [−219, −168]	11.8

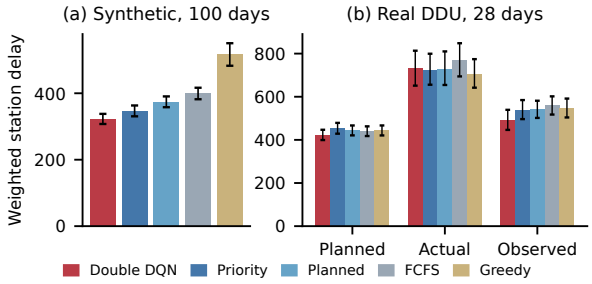


Fig. 2. Weighted station delay with 95% bootstrap intervals. (a) Synthetic test days. (b) Zero-shot replay of 28 real DDU days in three delay modes (x-axis, defined in the text; not the Planned policy).

on 82 of 100 days. All 500 episodes completed every train with zero safety violations. Against priority, its mean waiting per train is lower for premium (0.93 vs. 1.02 min), superfast, express and passenger trains but higher for low-weight freight (1.14 vs. 1.06), the trade-off encoded in W. The cost is plan adherence: 4.7× the platform changes of the planned policy. Greedy is worst, likely because one step of lookahead over noisy forecasts is too short-sighted.

Zero-shot transfer (unchanged checkpoint). *Seven-station network:* the junction sits in a double-track line with two-way traffic and six rule-based minor stations. Over 3 days the DQN beat FCFS on junction waiting (−43.2 [−54.0, −34.2]) and was inconclusive against priority (−11.7 [−28.0, +2.9]), yet had the lowest network lateness on every day.

Real data from Pt. Deen Dayal Upadhyaya Junction (DDU) (28 days, about 120 trains a day; delays from the RSTGCN dataset [6], stations from DataMeet [7]): all 420 episodes were safe (Fig. 2b). On the real timetable without delays (*Planned* mode) it beat all four heuristics (−32.5 [−50.7, −14.2] against priority). With delays taken from the previous station (*Actual* mode, our main mode) it was level with priority (+5.6 [−49.6, +70.7]). With delays recorded at DDU (*Observed* mode) it beat all four (−47.8 [−77.8, −15.5] against priority). *Fine-tuning* on 15 earlier days, with 3 for selection and 10 later test days, gave no significant gain in Actual mode (−5.8 [−55.1, +50.7]) but cut discretionary holds from 9.7 to 2.5 a day.

Conclusion. The DQN cut delay 7% below the best heuristic with zero safety violations and transferred safely without retraining. Limitations: one training run, no queue reordering, and only parity with priority in Actual mode.

REFERENCES

[1] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, 2016, pp. 2094–2100.

[2] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[3] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artif. Intell.*, vol. 112, pp. 181–211, 1999.

[4] P. A. Lopez *et al.*, "Microscopic traffic simulation using SUMO," in *Proc. IEEE ITSC*, 2018, pp. 2575–2582.

[5] M. Towers *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," arXiv:2407.17032, 2024.

[6] K. Chowdhury, P. Koley, A. Chakraborty, and S. Ghosh, "RSTGCN: Railway-centric spatio-temporal graph convolutional network for train delay prediction," *IEEE Trans. Intell. Transp. Syst.*, 2026.

[7] DataMeet, "Indian railways dataset (CC0)," https://github.com/datameet/railways.